PES UNIVERSITY

Machine Learning Laboratory 7

Multinomial Naive Bayes for Sentence Role Classification: From Scratch to a Bayes Optimal Approximation

Name: [YOUR NAME]

SRN: PES2UG24CS616

Course: Machine Learning (UE24CS352A), Semester V

Department of Computer Science & Engineering

Date: 14 October 2026

1. Introduction

This lab studies **sentence role classification** on biomedical text. Given one sentence from the abstract of a randomised controlled trial, the classifier must predict which section it came from: BACKGROUND, OBJECTIVE, METHODS, RESULTS or CONCLUSIONS. Solving this automatically makes unstructured abstracts searchable by rhetorical role.

The purpose of the lab is threefold. **Part A** implements a Multinomial Naive Bayes classifier from scratch in NumPy and verifies its parameters against scikit-learn's `MultinomialNB`. **Part B** investigates how smoothing strength (the parameter α) and the feature representation (raw counts versus TF-IDF, unigrams versus bigrams) change performance. **Part C** approximates the Bayes Optimal Classifier by combining four different models weighted by their posterior probability. Because the classes are imbalanced, **macro-averaged F1** is the primary metric throughout and all conclusions are drawn from it; accuracy is reported alongside.

2. Dataset Description

The data is the PubMed 20k RCT corpus, which ships pre-split into three files. The **training** split has 180,040 sentences and is used only to fit models; the **development** split has 30,212 sentences and is used to choose hyperparameters (α, n-gram range); the **test** split has 30,135 sentences and is scored only after every choice is fixed.

The five classes are strongly imbalanced (Table 1, Figure 1). METHODS and RESULTS together hold about 65% of the training sentences, whereas OBJECTIVE holds only 7.7%. A classifier that always predicted METHODS would reach 0.330 accuracy, which is why accuracy alone is a weak yardstick here.

Class	Training sentences	Share
BACKGROUND	21,727	12.1%
CONCLUSIONS	27,168	15.1%
METHODS	59,353	33.0%
OBJECTIVE	13,839	7.7%
RESULTS	57,953	32.2%

Table 1: Class distribution in the training split.

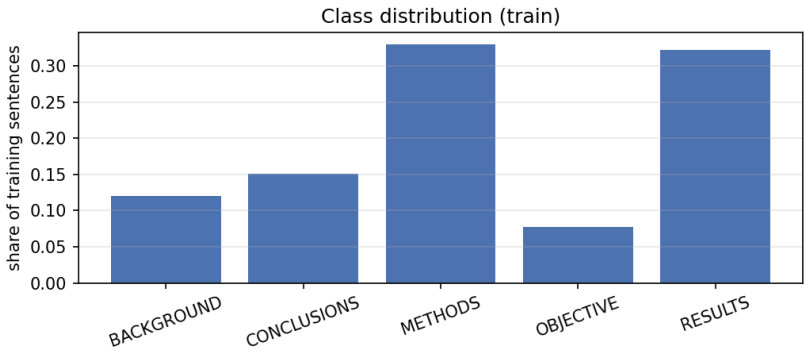


Figure 1: Share of training sentences per class.

Part C subsample size. The last three digits of the SRN (616) set the subsample to 10,000 + 616 = **10,616** training sentences. It is split 75/25 (stratified) into a fitting part of 7,962 sentences and a held-out part of 2,654 sentences.

Vocabulary size. Count features are built with `CountVectorizer` (lowercase, accent stripping, English stop words, `min_df=2`) fitted on the training split only, giving **V = 36,756** unigram features. The tuned

TF-IDF model with unigrams and bigrams has a vocabulary of 301,234 features, about 8.2 times larger.

3. Methodology

3.1 Multinomial Naive Bayes

Naive Bayes assigns a sentence to the class with the highest posterior. By Bayes' rule the evidence term P(x) is common to all classes and is dropped, and the words are assumed conditionally independent given the class:

$$c\blacksquare = argmax_c\, P(c) \prod_i P(w_i \mid c)^{x_i}$$

where x_i is the number of times word i occurs in the sentence. To avoid numerical underflow (a 25-word sentence multiplies about 25 probabilities of order 10^{-5}, which rounds to 0.0), all computation is done in log space:

$$score(c) = log\, P(c) + \sum_i x_i\, log\, P(w_i \mid c)$$

The class prior is the fraction of training *documents* in the class, $P(c) = N_c / N$. The word likelihood uses additive (Laplace/Lidstone) smoothing:

$$P(w_i \mid c) = (count(w_i, c) + \alpha) / (\sum_j count(w_j, c) + \alpha V)$$

Smoothing prevents a single word unseen in a class from driving that class's probability to zero. The αV term in the denominator is essential: V words each receive α extra mass, so the denominator must grow by αV for the distribution to sum to one.

3.2 Implementation

The class `NaiveBayesClassifier` stores `class_log_prior_` (shape n_classes) and `feature_log_prob_` (shape n_classes × V), both ordered to match `classes_`. Scoring a whole dataset is a single sparse matrix product, `X @ feature_log_prob_.T + class_log_prior_`, with no Python loop over documents; prediction takes the argmax over the class axis and maps the index back to a label.

3.3 Verification against scikit-learn

Before looking at any accuracy, the learned parameters were compared with those of `MultinomialNB(alpha=1.0)` fitted on the same count matrix. The maximum absolute difference was **1.55e-15** for the log priors and **1.78e-15** for the log likelihoods, both far below the required 10^{-10}, and the two models agreed on 100.00% of test predictions. This check tests the numbers themselves, which matters because a subtly wrong Naive Bayes still classifies: as a demonstration, omitting the αV term moved test accuracy only from 0.7357 to 0.7257 (macro-F1 from 0.6660 to 0.6607), a drop small enough to pass as a modelling choice, even though the outputs were no longer a probability distribution.

3.4 Experimental protocol

Experiment 1 sweeps α over {0.01, 0.1, 0.5, 1.0, 2.0} on the development set by macro-F1 and refits at the chosen value before touching the test set. Experiment 2 builds a TF-IDF + MultinomialNB pipeline, first with defaults and then tuned over n-gram range {(1,1), (1,2)} and α {0.1, 0.5, 1.0} with `GridSearchCV` (3-fold, macro-F1) on the development set, then refits the winner on the full training set. In Part C four hypotheses (Naive Bayes, Logistic Regression, Decision Tree, Random Forest, all on TF-IDF features) are fitted on the fitting part. Each one's posterior weight is estimated from its log-likelihood of the true labels on the held-out part (probabilities clipped at 10^{-12} before taking logs), converted with a softmax after subtracting the maximum. Two versions are computed: from the *mean* and from the *summed* log-likelihood (the exact Bayesian quantity, $P(h \mid D) \propto$ exp of the sum). The ensemble is a soft-voting classifier fitted on the full subsample.

4. Results and Analysis

4.1 Complete results table

Experiment	Accuracy	Macro-F1	Time (s)
TF-IDF + NB (tuned: ngram, alpha)	0.7525	0.6709	8.1
Custom NB (counts, alpha=1)	0.7357	0.6660	0.1
Custom NB (counts, tuned alpha=2.0)	0.7375	0.6632	0.2
Part C: LogisticRegression	0.7161	0.6285	1.0
BOC approx. (soft vote, summed-LL weights)	0.7161	0.6285	-
BOC approx. (soft vote, mean-LL weights)	0.7056	0.5984	4.1
TF-IDF + MultinomialNB (default)	0.7136	0.5911	3.5
Part C: NaiveBayes	0.6877	0.5610	0.6
Part C: RandomForest	0.5889	0.4124	1.6
Part C: DecisionTree	0.5131	0.3736	1.0

Table 2: Test-set accuracy and macro-F1 for every experiment, sorted by macro-F1. Part C rows use the 10,616-sentence subsample and are comparable with each other, not with the full-data rows. Times are machine-dependent.

The best model overall is the tuned TF-IDF + Naive Bayes (bigrams, α=0.1) with macro-F1 0.6709 and accuracy 0.7525, but it beats the plain from-scratch count model (0.6660) by only 0.0049 macro-F1.

4.2 Experiment 1: smoothing strength

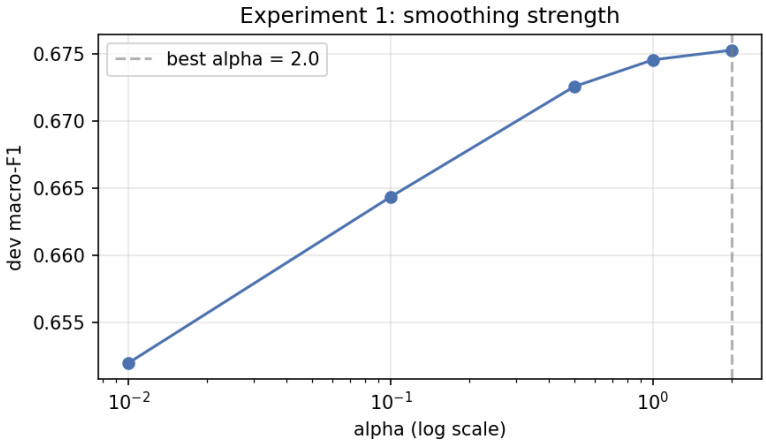


Figure 2: Development macro-F1 against α (log-scaled x-axis).

α	**0.01**	**0.1**	**0.5**	**1.0**	**2.0**
Dev macro-F1	0.6520	0.6644	0.6726	0.6746	0.6753

Very weak smoothing is clearly harmful: at α=0.01 a word never seen in a class is assigned a probability about 100 times smaller than a word seen once, so one rare or unseen word can outweigh several words of genuine evidence, and dev macro-F1 falls to 0.6520. The curve is nearly flat from α=0.5 upward (0.6726, 0.6746, 0.6753). The best value on dev was α=2.0, which sits at the edge of the grid and beats α=1 by only 0.0007. Refitted and scored once on test it gave 0.6632 macro-F1 against 0.6660 for α=1, so tuning near the optimum bought nothing; this is the expected behaviour when the dev curve is flat and shows why a dev winner is not guaranteed to be a test winner.

4.3 Experiment 2: counts versus TF-IDF

n-gram range	α	3-fold CV macro-F1 (dev)
(1, 2)	0.1	0.6235
(1, 1)	0.1	0.6036
(1, 1)	0.5	0.5597
(1, 2)	0.5	0.5361
(1, 1)	1.0	0.5001
(1, 2)	1.0	0.4614

Table 3: Grid search on the development set (best row first).

With defaults (α=1.0, unigrams) the TF-IDF model scored 0.5911 macro-F1, *worse* than raw counts (0.6660). The reason is the scale of the features. TF-IDF rows are L2-normalised, so a sentence contributes about 3.3 units of mass instead of its roughly 14 word counts, and each class's total mass is several times smaller (OBJECTIVE: 49,630 against 215,480 token counts). The smoothing mass αV = 36,756 is therefore 0.74 of OBJECTIVE's class mass under TF-IDF (0.19 for METHODS), against 0.17 and 0.04 under counts. α=1.0 flattens the word distributions much more heavily on TF-IDF, washing out class differences, most of all for small classes. The grid agrees: for unigrams CV macro-F1 falls steadily as α grows.

Tuning selected bigrams with α=0.1. Bigrams raised the CV macro-F1 from 0.6036 to 0.6235 at α=0.1, because fixed phrases are kept together rather than split into independent words. The cost is a vocabulary of 301,234 features (about 8 times larger), a sparser model, and a slower fit (8.1 s against 3.6 s for the count pipeline in this run). On test the tuned model reached 0.6709 macro-F1 and 0.7525 accuracy.

4.4 Confusion matrices and which classes are confused

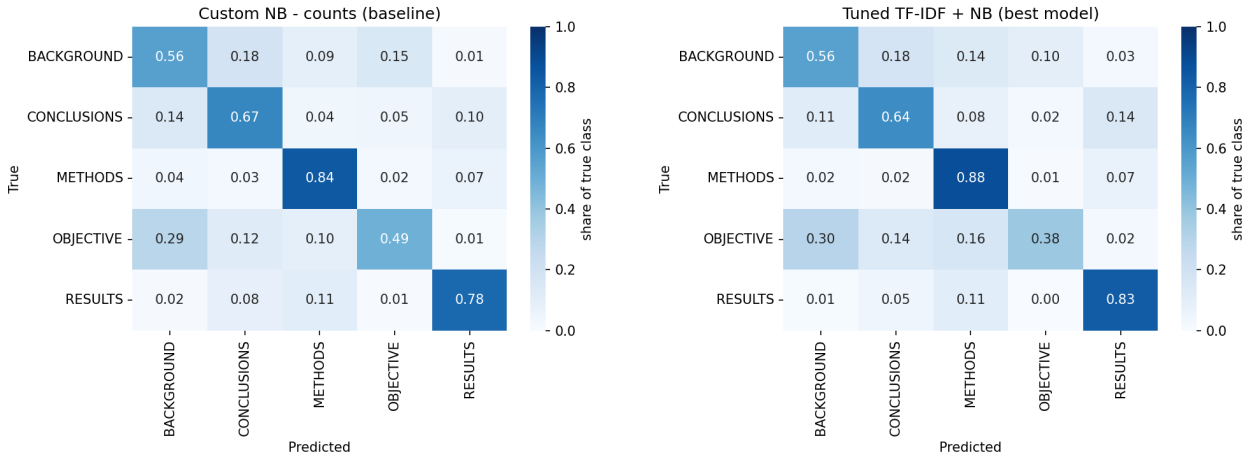


Figure 3: Row-normalised confusion matrices on the test set. Left: baseline (count NB, α=1). Right: best model (tuned TF-IDF + NB). Each row shows where the sentences of one true class were sent.

Class	Baseline P / R / F1	Tuned TF-IDF P / R / F1	Support
BACKGROUND	0.52 / 0.56 / 0.54	0.57 / 0.56 / 0.56	3,621
CONCLUSIONS	0.60 / 0.67 / 0.63	0.65 / 0.64 / 0.65	4,571
METHODS	0.82 / 0.84 / 0.83	0.79 / 0.88 / 0.84	9,897
OBJECTIVE	0.52 / 0.49 / 0.50	0.61 / 0.38 / 0.47	2,333
RESULTS	0.86 / 0.78 / 0.82	0.84 / 0.83 / 0.84	9,713

Table 4: Per-class precision, recall and F1 on the test set.

Analysis. In the baseline, the largest off-diagonal entries are OBJECTIVE predicted as BACKGROUND (0.29 of true OBJECTIVE sentences), BACKGROUND predicted as CONCLUSIONS (0.18) and BACKGROUND predicted as OBJECTIVE (0.15); CONCLUSIONS predicted as BACKGROUND (0.14) is next. These are the classes that share vocabulary. OBJECTIVE and BACKGROUND sentences both name the condition, the patient population and the intervention, and BACKGROUND and CONCLUSIONS sentences are both general statements without numbers. The words that separate them (cue words such as *evaluate* or *investigate*) are few compared with the shared topical words, and because Naive Bayes simply adds up per-word evidence, the shared words outvote the cues. The pattern is therefore overlapping vocabulary between particular pairs of classes, not a generally weak model.

METHODS (recall 0.84) and RESULTS (0.78) are well separated because their vocabulary is distinctive (dosages, durations, randomisation terms versus numbers, percentages, *significantly*); their main mutual confusion is only 0.11 (RESULTS to METHODS). OBJECTIVE is the weakest class (F1 0.50), matching the expectation from the class distribution, with BACKGROUND next (F1 0.54). The tuned TF-IDF model improves METHODS and RESULTS recall (to 0.89 and 0.83) but its OBJECTIVE recall falls to 0.38 while precision rises from 0.52 to 0.61: it became more cautious about OBJECTIVE rather than uniformly better, and 0.30 of true OBJECTIVE sentences are still sent to BACKGROUND. The remaining error is therefore concentrated in the same overlapping pairs, which is why even the best model gains only a few points of macro-F1 over the baseline.

4.5 Part C: the Bayes Optimal Classifier approximation

Hypothesis	Mean log-lik	Held-out acc	Weight (mean)	Sum log-lik	Weight (sum)	Test macro-F1
NaiveBayes	-0.9065	0.6507	0.3254	-2406.0	5.2e-45	0.5610

Hypothesis	Mean log-lik	Held-out acc	Weight (mean)	Sum log-lik	Weight (sum)	Test macro-F1
LogisticRegression	-0.8681	0.6839	0.3382	-2304.0	1.0000	0.6285
DecisionTree	-2.2598	0.5023	0.0841	-5997.5	0.0000	0.3736
RandomForest	-1.1607	0.5780	0.2524	-3080.6	0.0000	0.4124

Table 5: Posterior weights from the held-out part (n = 2,654) and test performance of each hypothesis and of the ensembles.

Did the ensemble beat its members? No. The mean-weight soft vote scored 0.5984 macro-F1: above Naive Bayes (0.5610), Random Forest (0.4124) and Decision Tree (0.3736) but 0.030 below Logistic Regression (0.6285). The mean log-likelihoods differ by only a few tenths, so the weights are close to equal (LR 0.338, NB 0.325, RF 0.252, DT 0.084) and the vote effectively averages in the weaker models: Random Forest carries a quarter of the weight despite scoring 0.41 alone, which drags the ensemble under its best member.

Why the exact weights collapse. The summed log-likelihood is the mean multiplied by n = 2,654, so a per-sentence gap of about 0.04 between Logistic Regression and Naive Bayes becomes 102 log-units, a likelihood ratio of about e^{102}. The exact posterior is essentially a point mass: LR receives weight 1.000, NB 5e-45 and the others underflow to 0. With only a handful of hypotheses and a few thousand held-out examples, the Bayes Optimal Classifier thus degenerates into plain model selection; the benefit of averaging appears only when many hypotheses carry comparable posterior mass, which needs a rich hypothesis space or very little data. Here the summed weights scored better (0.6285 macro-F1, 0.7161 accuracy, identical to LR alone) than the mean weights (0.5984, 0.7056), because LR really was the best member, as the test scores confirm. The test-set confusion matrix of the mean-weight ensemble (not shown) also reveals a weakness of the weaker members: its OBJECTIVE recall is only 0.28, and 0.34 of true OBJECTIVE sentences are sent to METHODS.

5. Conclusion

Implementation. The from-scratch multinomial Naive Bayes matches scikit-learn to about 10^{-15} in both priors and likelihoods and reaches 0.7357 accuracy and 0.6660 macro-F1 on the test set at $\alpha=1$. The parameter comparison, not the accuracy, is what establishes correctness, since removing the αV term costs only about one point.

Smoothing. Too little smoothing hurts (dev macro-F1 0.6520 at $\alpha=0.01$), but from $\alpha=0.5$ to 2.0 the curve is flat within 0.0027. Tuning α on dev gave no test improvement (0.6632 against 0.6660), so smoothing is a matter of avoiding the extreme, not of fine-tuning.

Feature representation. This changed macro-F1 the most. Untuned TF-IDF was 0.075 worse than counts because L2-normalised values make $\alpha=1$ far too strong; tuning α to 0.1 and adding bigrams recovered it to 0.6709, a net gain of only 0.005 over plain counts at the price of an 8-fold larger vocabulary. The plain count-based model offers the best return for the effort.

Ensemble. The posterior-weighted ensemble (0.5984) did not improve on its best individual member, Logistic Regression (0.6285). Mean-log-likelihood weights are nearly uniform and let weak models dilute the vote, while the exact summed weights collapse onto one model and reduce the Bayes Optimal approximation to model selection. Across all experiments the remaining errors are concentrated in the overlapping OBJECTIVE, BACKGROUND and CONCLUSIONS classes, which no choice of smoothing, features or model combination in this lab resolved.